

Software Development Fundamentals (SDF) – I ODD 2023



Recursive Functions in C

Jaypee Institute of Information Technology (JIIT), Noida

Recursive Functions

- In C, it is possible for the functions to call themselves .
- A function is called 'recursive' if a statement within the body of a function calls the same function.
- Sometimes called 'circular definition', recursion is thus the process of defining something in terms of itself.

Two cases

- A recursive definition has two parts:
 - One or more recursive cases where the function calls itself
 - One or more base cases that returns a result without a recursive call
- There **must** be at least one base case
- Every recursive case **must** make progress towards a base case

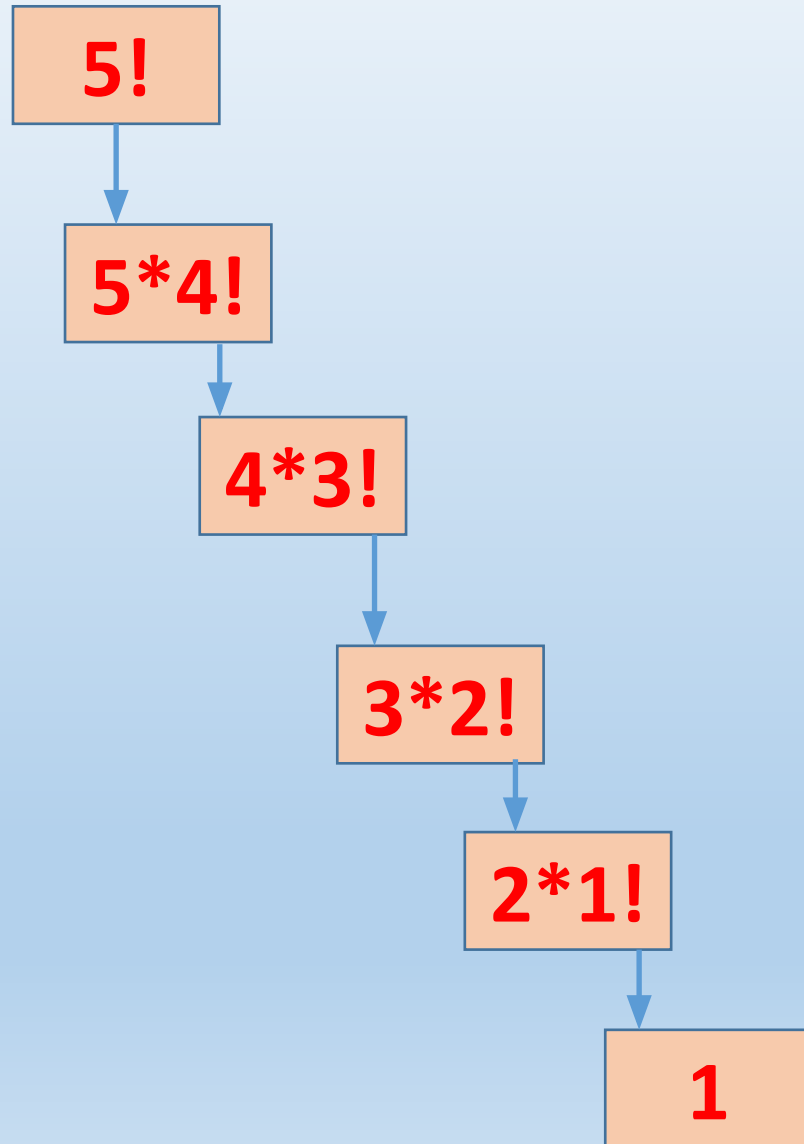
Example 1: Recursively Calculating Factorials

Factorial of a number **n!** is defined mathematically as :

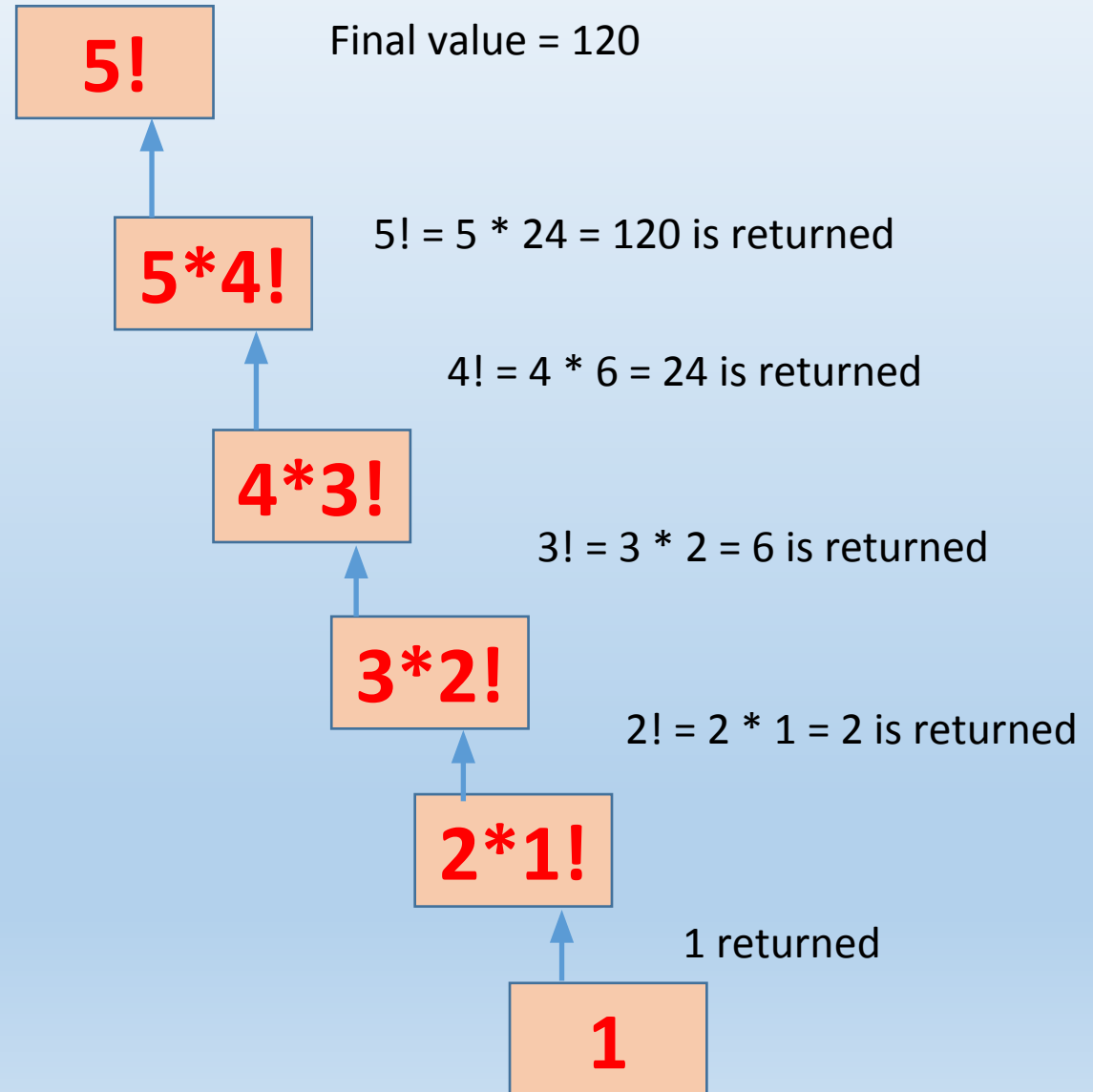
$$\mathbf{n! = \begin{cases} 1 & \mathbf{n = 0} \\ \mathbf{n * (n - 1)!} & \mathbf{otherwise} \end{cases}}$$

For calculating **n!** first **(n-1)!** should be calculated and for that **(n-2)!** should be computed and so on ...

(a) Sequence of recursive calls



(b) Values returned from each recursive call.



Program to calculate factorial value

```
main( )
{
int a, fact ;
printf ( "\nEnter any number " );
scanf ( "%d", &a );
fact = rec ( a );
printf ( "Factorial value = %d", fact );
}
rec ( int x )
{
int f ;
if ( x == 1 )
return ( 1 );
else
f = x * rec ( x - 1 );
return ( f );
}
```

output :

Enter any number 1

Factorial value = 1

Enter any number 2

Factorial value = 2

Enter any number 3

Factorial value = 6

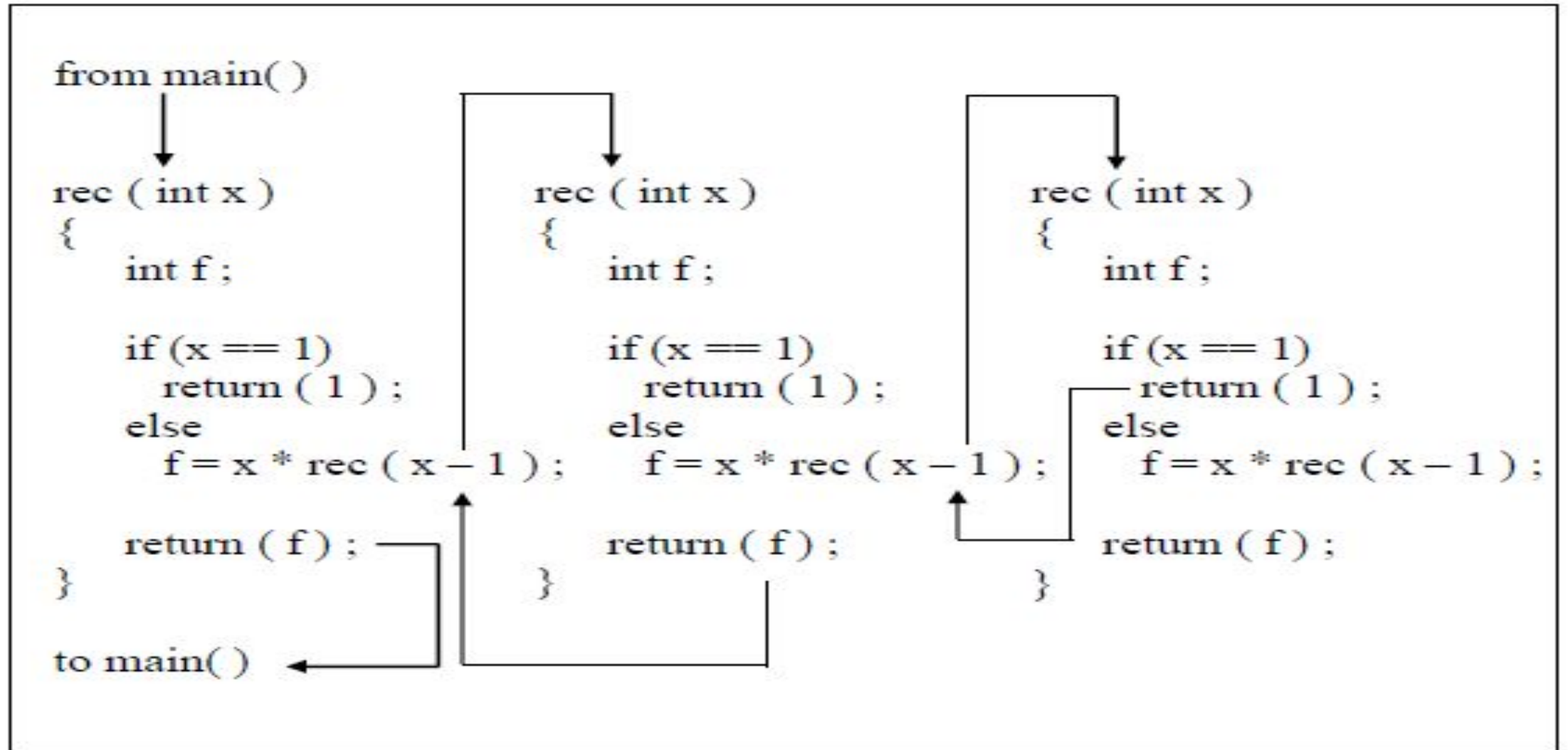
Enter any number 5

Factorial value = 120

Recursive factorial function

- when the number entered through **scanf()** is 1 , The value of **a** (i.e. 1) is copied into **x**. Since **x** turns out to be 1 the condition **if (x == 1)** is satisfied and hence 1 is returned through the **return** statement.
- When the number entered through **scanf()** is 2, the **(x == 1)** test fails, so we reach the statement,
$$f = x * \text{rec} (x - 1) ;$$
- We multiply **x** by **rec (x - 1)**. Since the current value of **x** is 2, it is same as saying that we must calculate the value $(2 * \text{rec} (1))$.

Control flows during successive recursive calls

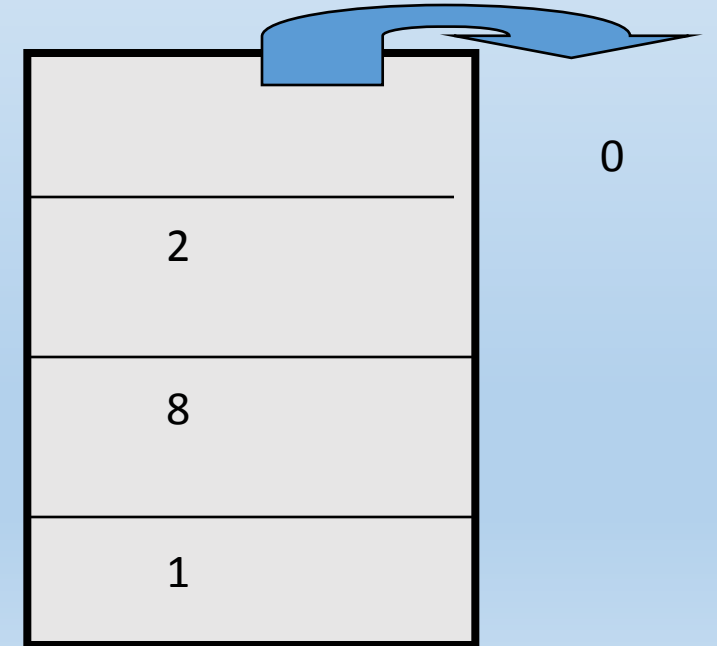
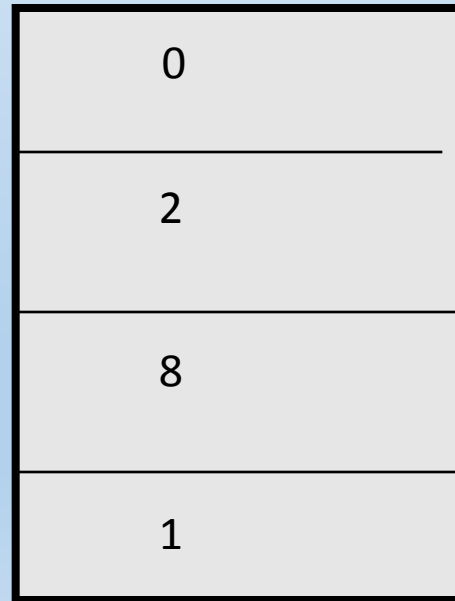
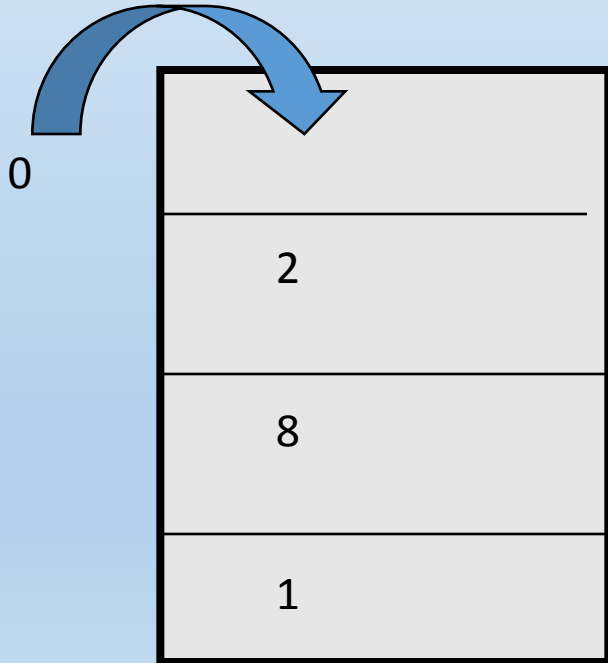


LOGIC

- When a recursive function is executed, the recursive calls are not executed immediately. Rather, **they are placed on a stack** until the **condition that terminates the recursion is encountered**.
- The function calls are then executed in **reverse order**, as they are popped off the stack.

STACK

- A STACK is a **LAST-IN, FIRST-OUT** data structure in which successive data items are pushed down upon the preceding data items. **The data items are removed (popped off) from the stack in REVERSE order.**



Factorial

- `x = factorial(3)`

3 is put on stack as n

- `int factorial(int n) { //n=3`

- `int r = 1;`

r is put on stack with value 1

- `if (n <= 1) return r;`

- `else {`

- `r = n * factorial(n - 1);`

- `return r;`

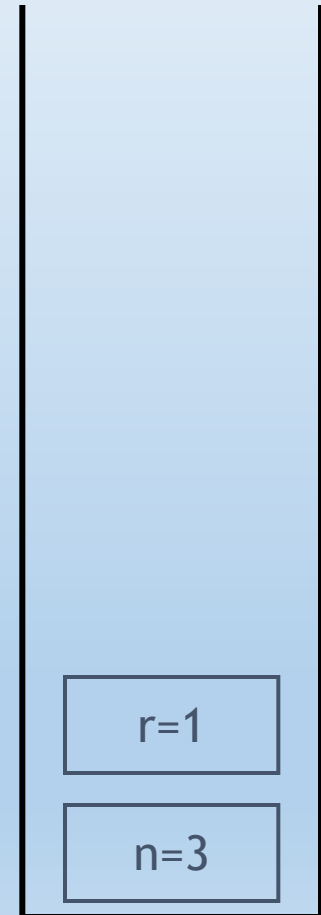
- `}`

- `}`

All references to `r` use this `r`

All references to `n` use this `n`

Now we recur with 2...



Factorial

- $r = n * \text{factorial}(n - 1);$

2 is put on stack as n

- `int factorial(int n) { // n=2`

- `int r = 1;`

r is put on stack with value 1

- `if (n <= 1) return r;`

- `else {`

- `r = n * factorial(n - 1);`

- `return r;`

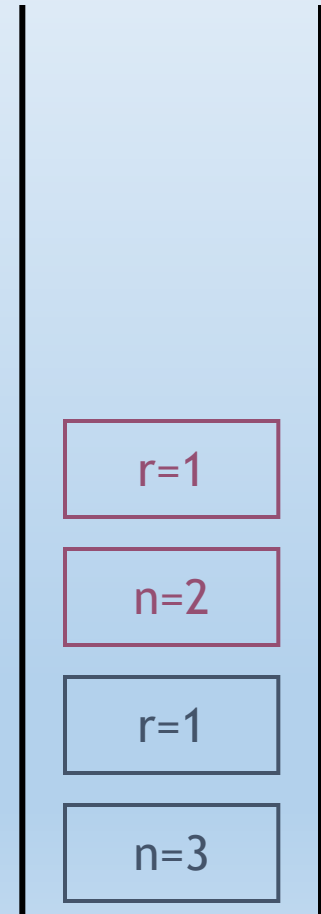
- `}`

- `}`

Now using this r

And this n

Now we recur with 1...



Factorial

- $r = n * \text{factorial}(n - 1);$

1 is put on stack as n

- `int factorial(int n) { // n=1`

`int r = 1;`

r is put on stack with value 1

`if (n <= 1) return r;`

`else {`

`r = n * factorial(n - 1);`

`return r;`

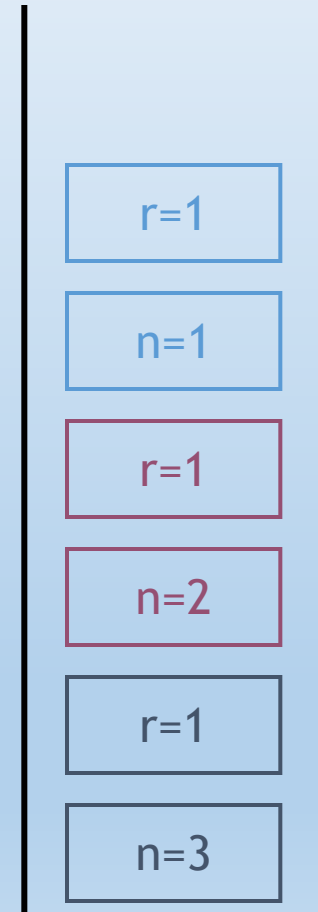
`}`

`}`

Now using this r

And this n

Now we pop r and n off the stack
and return 1 as factorial(1)

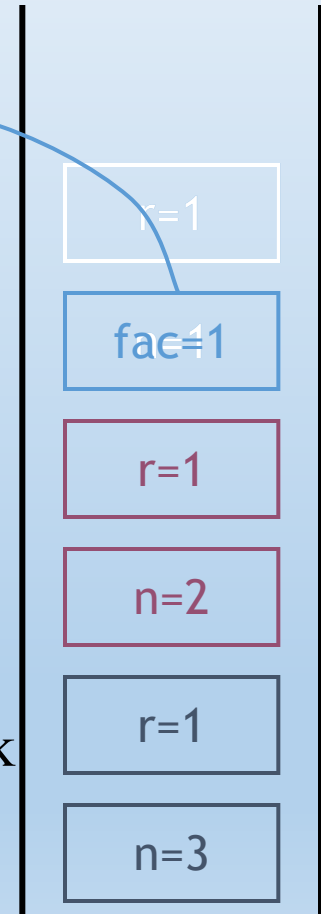


Factorial

- $r = n * \text{factorial}(n - 1);$

- ```
int factorial(int n) {
 int r = 1;
 if (n <= 1) return r;
 else {
 r = n * factorial(n - 1);
 return r;
 }
}
```

Now using this r  
And this n



Now we pop r and n off the stack  
and return 1 as factorial(1)

# Factorial

- $r = n * \text{factorial}(n - 1);$

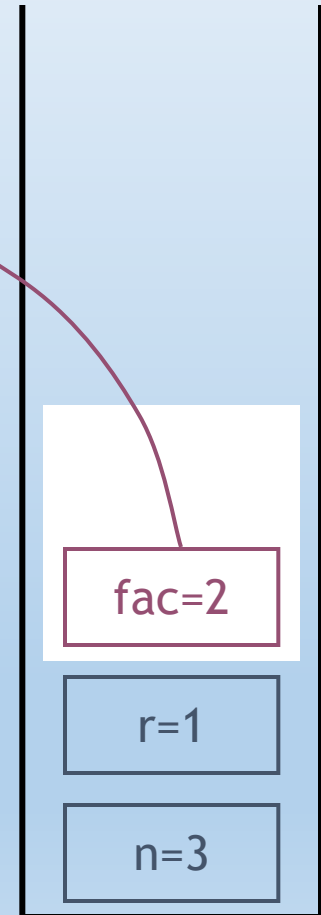
- ```
int factorial(int n) {  
    int r = 1;  
    if (n <= 1) return r;  
    else {  
        r = n * factorial(n - 1);  
        return r;  
    }  
}
```

1
2 * 1 is 2;
Pop r and n;

Return 2

Now using this r

And this n



Factorial

- `x = factorial(3)`

- ```
int factorial(int n) {
 int r = 1;
 if (n <= 1) return r;
 else {
 r = n * factorial(n - 1);
 return r;
 }
}
```

3 \* 2 is 6;  
Pop r and n;

Return 6

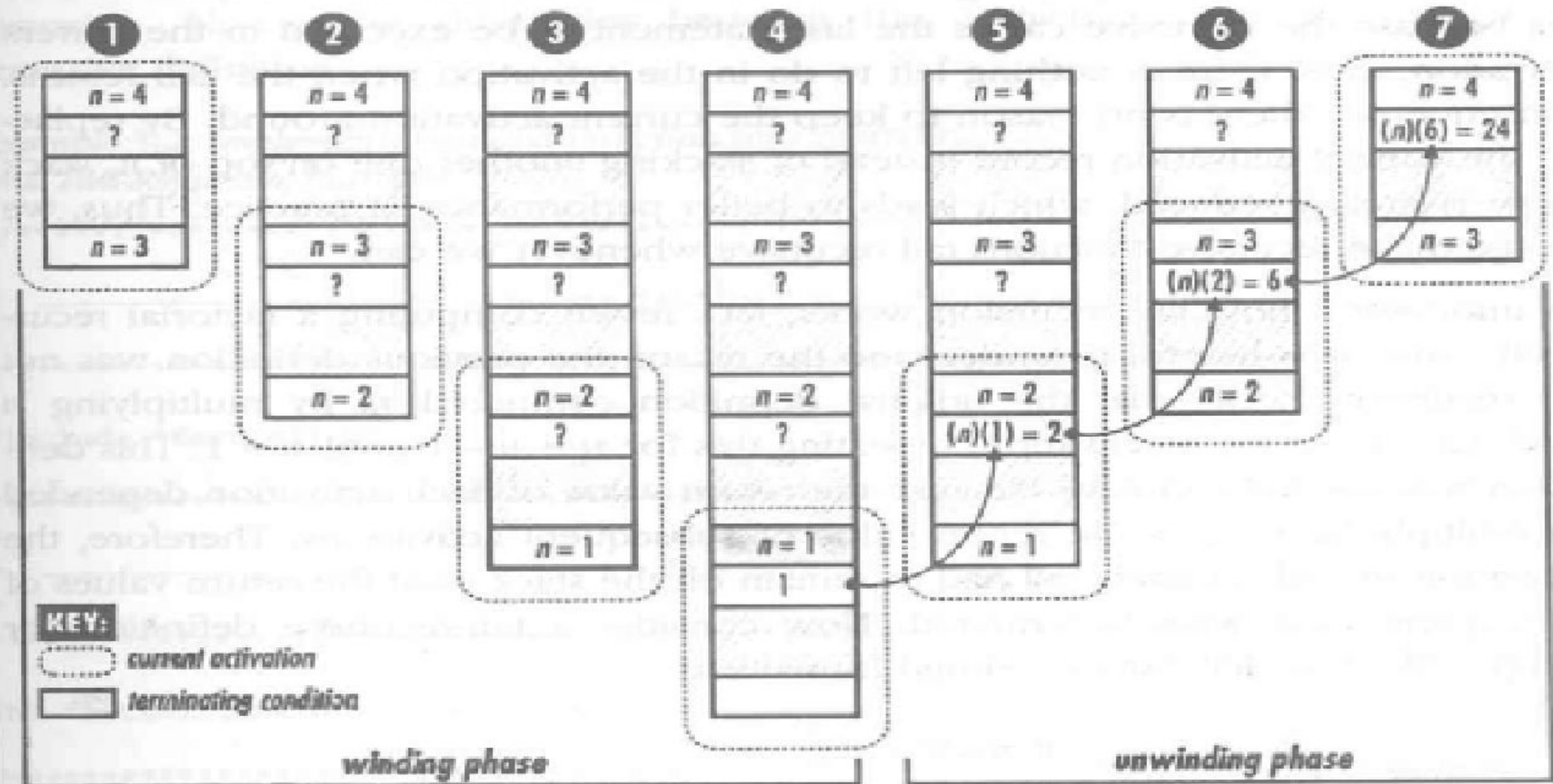
Now using this r

And this n

r=1

fac=6

# The stack of C program while computing 4! recursively



## Example 2: Fibonacci Series using recursion

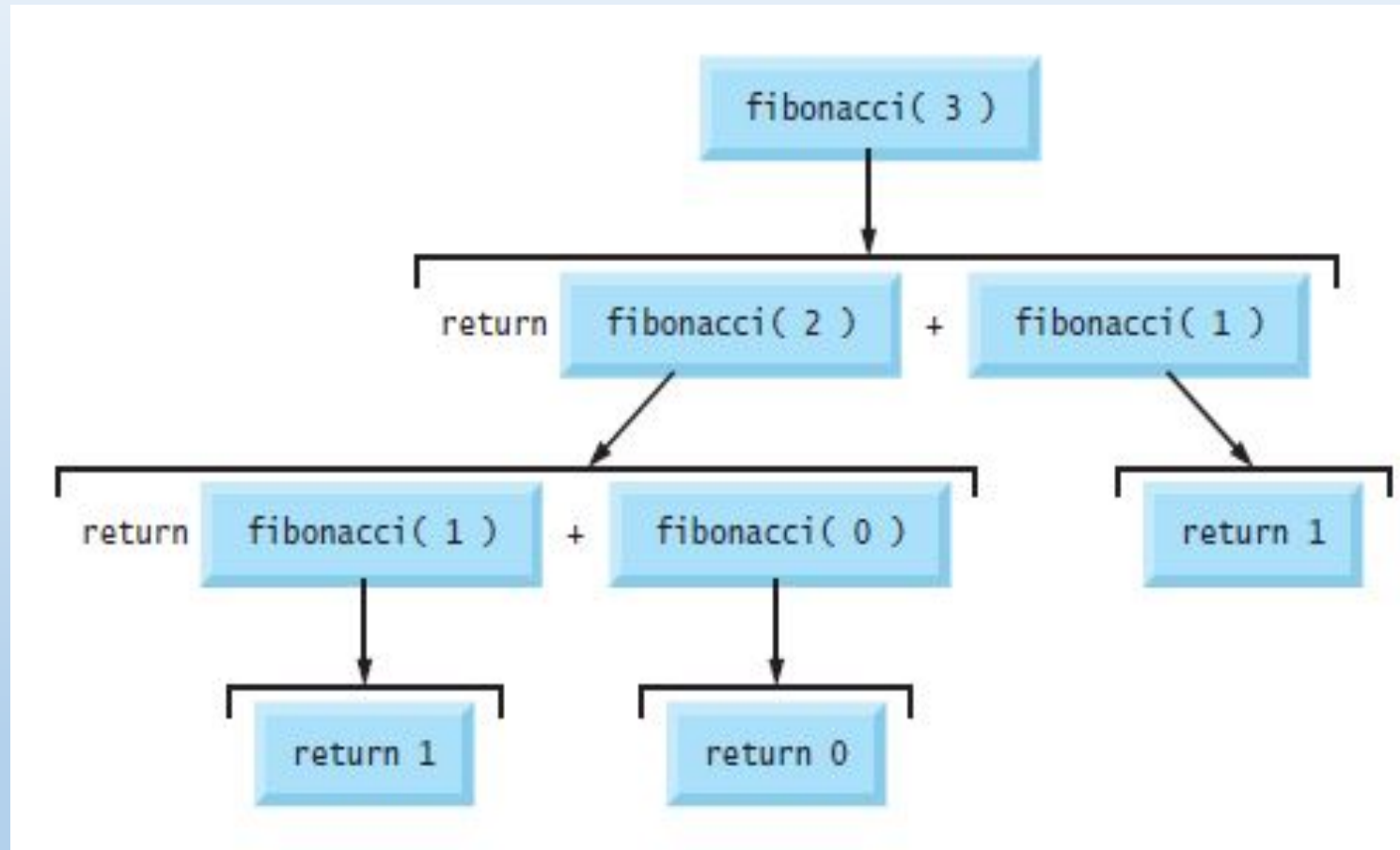
- The Fibonacci series  
0, 1, 1, 2, 3, 5, 8, 13, 21, ...  
begins with 0 and 1 and has the property that each subsequent Fibonacci number is the sum of the previous two Fibonacci numbers.
- The Fibonacci series may be defined recursively as follows:

$$\text{fibonacci}(0) = 0$$

$$\text{fibonacci}(1) = 1$$

$$\text{fibonacci}(n) = \text{fibonacci}(n - 1) + \text{fibonacci}(n - 2)$$

## Set of recursive calls for fibonacci ( 3 )



# Program to Generate the Fibonacci series for a given number

```
#include <stdio.h>
int fibonacci(int i) {
 if(i == 0) {
 return 0;
 }
 if(i == 1) {
 return 1;
 }
 return fibonacci(i-1) + fibonacci(i-2);
}
int main() {
 int i;
 for (i = 0; i < 10; i++) {
 printf("%d\t\n", fibonacci(i));
 }
 return 0;
}
```

output :

0

1

1

2

3

5

8

13

21

34

## Example 3: C Program to calculate sum of numbers 1 to N using recursion

```
int calculateSum(int num);

void main() {
 int i, num;
 int result;
 printf("Input a number : ");
 scanf("%d", &num);
 result = calculateSum(num);
 printf("\nSum of Number From 1 to %d : %d", num, result);
}

int calculateSum(int num) {
 int res;
 if (num == 1) {
 return (1);
 } else {
 res = num + calculateSum(num - 1);
 }
 return (res); }
```

#### Example 4: Find Sum of Digits of the Number using Recursive Function

```
#include<stdio.h>

int calsum(int num) {
 int rem, sum;
 if (num != 0) {
 rem = num % 10;
 sum = sum + rem;
 calsum(num / 10);
 }
 return sum;
}

int main() {
 int num, val;
 printf("\nEnter a number: ");
 scanf("%d", &num);
 val = calsum(num);
 printf("\nSum of the digits of %d is : %d", num, val);
 return 0;
}
```

### Example 5: WAP to compute GCD of Two Numbers using Recursion

```
#include <stdio.h>
int hcf(int n1, int n2);
int main()
{
 int n1, n2;
 printf("Enter two positive integers: ");
 scanf("%d %d", &n1, &n2);

 printf("G.C.D of %d and %d is %d.", n1, n2, hcf(n1,n2));
 return 0;
}

int hcf(int n1, int n2)
{
 if (n2 != 0)
 return hcf(n2, n1%n2);
 else
 return n1;
}
```

# Exercise

Write a recursive function for the Ackermann's Function

$$A(x, y) \equiv \begin{cases} y + 1 & \text{if } x = 0 \\ A(x - 1, 1) & \text{if } y = 0 \\ A(x - 1, A(x, y - 1)) & \text{otherwise.} \end{cases}$$

# Solution

```
int ackermann(int m, int n)
{
 if (m == 0) {
 return n + 1; }
 else if ((m > 0) && (n == 0)) {
 return ackermann(m-1, n); }
 else if ((m > 0) && (n > 0)) {
 return ackermann(m-1, ackermann(m,n-1)); }
 else {
 return 0; }
}
```

End of Lecture